

Sistema di Logistica Droni Resiliente con AI Agentica

Lorenzo Amorosa
Edoardo Buttazzi
Gregorio Cavulla

Indice

1	Introduzione e Definizione del Servizio	3
1.1	Dominio Operativo	3
1.2	Stakeholder e Utenti Target	4
1.3	Analisi del Workload	5
2	Architettura del Sistema e Osservabilità	7
2.1	Design a Microservizi	7
2.2	Simulazione del Mondo Fisico (Digital Twin)	8
2.3	Osservabilità (Observability Strategy)	8
3	Agentic AI e Design MCP (Model Context Protocol)	10
3.1	Il Limite dell’Automazione Deterministica	10
3.2	Architettura MCP	10
3.3	Ruolo 1: The Health Agent	11
3.4	Ruolo 2: The Logistic Agent	11
3.5	Orchestrazione Dinamica	12
4	Operational Safety Policy e Agent Guardrails	13
4.1	Principio del Minimo Privilegio (Least Privilege)	13
4.2	Protezione contro Loop e Consumo Token	14
4.3	Protezione contro scollegamento backend	14
4.4	Idempotenza delle Azioni	14
4.5	Human-In-The-Loop (HITL) ed Escalation	15
5	Meccanismi di Reliability e Scalabilità	17
5.1	Resilienza di Rete	17
5.2	Graceful Degradation del Server Centrale	17
5.3	Scalabilità Dinamica e Safety Limits K8s	18
6	Chaos Engineering e Test di Fallimento	19
6.1	Metodologia	19
6.2	Esperimento 1: Primary Database Outage	19
6.3	Esperimento 2: Network Resilience (MQTT Broker Crash)	20
6.4	Esperimento 3: Load Spike e Scaling Agentico	20
6.5	Riepilogo Metriche	20
6.6	Lezioni Apprese	21

7	Analisi Economica: Costi, ROI e ROA (Return on Agent)	22
7.1	Service Level Objectives (SLOs) e Costo del Downtime	22
7.2	Valutazione del Return on Agent (ROA) e Costi	23
7.2.1	Strategie di Ottimizzazione dei Costi Operativi (OPEX)	23
7.2.2	Mitigazione del Rischio e Preservazione del Capitale (CAPEX) . . .	23
7.3	Analisi dei Trade-Off	24
8	Conclusioni	25

Capitolo 1

Introduzione e Definizione del Servizio

1.1 Dominio Operativo

Il progetto si colloca all'intersezione tra due domini critici identificati dalle specifiche di progetto: *Transportation* e *Industrial / IoT Monitoring*. Nello specifico, il sistema implementa un'architettura logistica autonoma per la gestione di una flotta di droni adibita alla consegna urbana di *payload* critici (come forniture mediche d'emergenza, organi per trapianti o componentistica industriale ad alta priorità). In un servizio "high-stakes" come questo l'efficienza e l'affidabilità del sistema sono fondamentali per la buona riuscita di missioni.

Il Contesto Fisico e Simulato

Il sistema opera all'interno di un'area metropolitana modellata metricamente, con un raggio d'azione di 5000 metri (5 km) attorno a un HUB logistico centrale posizionato alle coordinate base $[0.0, 0.0]$. L'ambiente operativo è altamente dinamico e soggetto a vincoli fisici realistici che l'Intelligenza Artificiale deve interpretare e gestire:

- **Gestione del Payload:** Gli ordini generati presentano un peso variabile (da 0.5 kg a 5.0 kg) e tre livelli di priorità (*low*, *normal*, *high*).
- **Fisica del Volo:** I droni non sono entità ideali, ma agenti fisici il cui stato viene modellato in base allo sforzo. Il consumo della batteria e la velocità di crociera (che varia dai 50 km/h agli 80 km/h) sono calcolati dinamicamente in proporzione diretta al peso del carico trasportato. Il drone può trovarsi in stato IDLE, IN DELIVERY, RETURNING e MAINTENANCE.
- **Degrado Meccanico (Wear):** Ogni missione accumula una percentuale di usura (da 0% a 100%) basata sulla distanza percorsa. Il superamento di soglie critiche richiede interventi di manutenzione preventiva e ragionamenti per evitare schianti catastrofici.

La Necessità di un Approccio Agentico

In un dominio tradizionale, l'assegnazione degli ordini avverrebbe tramite semplici code FIFO (First-In, First-Out) o algoritmi deterministici di routing. Tuttavia, l'elevata varia-

bilità delle condizioni (es. un pacco da 4.5 kg ad altissima priorità, ma con a disposizione solo droni con batteria bassa o con usura avanzata) rende l'automazione rigida inefficiente e pericolosa. Il sistema delega questo triage a un layer operativo basato su Agentic AI. L'obiettivo del dominio applicativo è dimostrare come l'IA possa orchestrare un'infrastruttura IoT in tempo reale, bilanciando il rischio operativo, l'urgenza delle consegne e la salute della flotta.

1.2 Stakeholder e Utenti Target

Per comprendere appieno i requisiti di affidabilità, sicurezza e scalabilità dell'architettura, è necessario definire gli attori che interagiscono con i vari componenti. Gli attori si dividono in due macro-categorie: gli *stakeholder* (coloro che hanno un interesse diretto nella gestione e sostenibilità economica della piattaforma) e gli *utenti target* (i beneficiari del servizio logistico).

Stakeholder

1. **Operatori di Flotta e Supervisor Umani (Human-in-the-Loop):** Sono gli utenti responsabili della supervisione strategica e della sicurezza operativa del sistema. Interagiscono direttamente con il gateway di sicurezza per validare o respingere le azioni ad alto impatto generate autonomamente dall'Intelligenza Agantica. Tra queste azioni figurano lo scaling della flotta oltre le soglie di sicurezza automatizzate o l'invio di comandi MQTT critici. Gli operatori monitorano inoltre lo stato in tempo reale dei droni e degli ordini tramite la dashboard web.
2. **Ingegneri di Infrastruttura e DevOps/SRE Team:** Professionisti incaricati di garantire l'alta disponibilità e l'affidabilità dell'architettura. Il loro obiettivo principale è monitorare la salute dei nodi e dei pod Kubernetes che ospitano i droni simulati. Verificano inoltre il corretto instradamento dei messaggi sul broker Mosquitto e assicurano la continuità della persistenza dei dati su InfluxDB. Intervengono in caso di escalation infrastrutturale qualora gli agenti riproducano log di errore legati a guasti fisici o problemi di risoluzione DNS non mitigabili autonomamente.
3. **Business Owner e Controller Finanziari:** Soggetti interessati alla sostenibilità economica e all'efficienza della piattaforma. Valutano il bilanciamento tra investimenti infrastrutturali (CAPEX) e costi operativi (OPEX). Pongono particolare attenzione al monitoraggio del tetto massimo di spesa legato al consumo di token per l'inferenza dei modelli LLM (*Agent-related cost ceiling*). Il loro KPI di riferimento è il *Return on Agent* (ROA). Questo indicatore misura il risparmio derivante dalla prevenzione dei guasti hardware tramite manutenzione predittiva rispetto al costo vivo delle chiamate API dell'IA.

Utenti Target (Clienti del Servizio)

- **Entità Mittenti (Hub Medici, E-Commerce Critici, Hub Industriali):** Rappresentano i clienti primari del software logistico. Sono responsabili dell'immissione asincrona di richieste di consegna all'interno della rete. Questi utenti richiedono il rigido rispetto dei *Service Level Objectives* (SLO) concordati. Esigono che gli ordini

ad alta priorità (*high priority*) vengano presi in carico il prima possibile e consegnati con tempistiche accettabili anche in scenari di forte congestione della coda (*load spikes*).

- **Destinatari Finali (Laboratori di Analisi, Linee di Produzione JIT):** I beneficiari ultimi del payload trasportato dai velivoli autonomi. Nel contesto di spedizioni urgenti o industriali (*Just-In-Time*), il destinatario dipende strettamente dalla precisione e dalla resilienza del sistema. Per questa categoria di utenti, la minimizzazione del downtime e l'eliminazione dei fallimenti (es. schianto del drone per esaurimento della batteria in volo) rappresentano requisiti di affidabilità critici.

1.3 Analisi del Workload

L'architettura è di tipo *event-driven*, e sottoposta a un carico di lavoro ibrido, caratterizzato dalla coesistenza di flussi di dati continui (streaming IoT) e da eventi asincroni variabili (richieste di business e cicli di ragionamento AI). Per dimensionare correttamente i meccanismi di reliability e scalabilità, il workload è stato scomposto in tre sezioni principali:

1. Telemetria IoT ad Alta Frequenza (Constant Load)

Il carico di base dell'infrastruttura è generato dalla flotta. Ogni nodo (pod) del `drone_simulator` calcola costantemente il proprio movimento e il decadimento della batteria, trasmettendo un payload JSON contenente coordinate spaziali, stato operativo e usura. Ciascun drone pubblica un aggiornamento sul broker MQTT (topic `telemetry/drones`) con una frequenza di un messaggio ogni 2 secondi. Questo genera un throughput in ingresso costante, che il `central_server` deve prima elaborare in RAM per mantenere aggiornata la Dashboard real-time, e successivamente persistere su InfluxDB strutturando i dati come serie temporali.

2. Eventi di Business asincroni (Spiky Load)

Il secondo vettore è rappresentato dalle richieste di consegna generate dai nodi cliente (`client_simulator.py`). Questo traffico è composto dagli ordini generati casualmente e emessi sul topic `business/ordini/nuovi`. Allo stesso tempo il sistema è soggetto a picchi anomali in scenari di stress testing che forzano l'intervento reattivo del sistema.

3. Carico di Inferenza e Polling Agentico (Read-Heavy Spikes)

Una peculiarità delle architetture che sfruttano AI Agentica è il carico indiretto generato dall'Intelligenza Artificiale stessa sull'infrastruttura sottostante. L'orchestratore centrale (`logistic_ai.brain.py`) si occupa di risvegliare gli agenti e ogni risveglio genera varie operazioni in lettura (Polling):

- Interrogazione intensiva di InfluxDB per estrarre la finestra temporale (time-window) degli ultimi 5-60 minuti di telemetria e ordini.
- Chiamate alle API di Kubernetes per ottenere lo stato dei Pod attivi in tempo reale.

Questo workload "a raffiche" richiede che il database e le API di K8s rispondano con latenze minime, poiché un ritardo nella fase di *information gathering* prolungherebbe i tempi di inferenza del Large Language Model, violando i *Service Level Objectives* (SLO) di reattività del sistema.

Capitolo 2

Architettura del Sistema e Osservabilità

Nel progetto l'architettura è basata su Kind (Kubernetes IN Docker), viene utilizzato per avviare cluster Kubernetes locali e leggeri dentro dei container di Docker replicando molti comportamenti di un cluster reale (scheduler, networking, controller) in un ambiente contenuto e facilmente ricreabile, ideale per validazione e test prima di un eventuale rilascio.

Kubernetes è un orchestratore di container che gestisce deployment, scaling e resilienza delle applicazioni: organizza i container in Pod, mantiene lo stato desiderato con Deployment e ReplicaSet, espone servizi con Service e fornisce autoscaling e self-healing. Aspetti che nel progetto vengono utilizzati tramite invocazione delle funzioni di `kubectl`.

Docker è una piattaforma di containerizzazione che confeziona applicazioni e dipendenze in immagini portabili, garantisce isolamento, avvio rapido e coerenza tra ambienti. Funge da base su cui costruire e distribuire le immagini eseguite da kind (o Kubernetes).

2.1 Design a Microservizi

Nel progetto abbiamo adottato un design a microservizi che separa chiaramente responsabilità e domini di failure in tre blocchi indipendenti: Data Layer (InfluxDB) dedicato alla persistenza di metriche/serie temporali, Message Broker (MQTT/Mosquitto) come canale asincrono di scambio eventi, e Logic Layer (Server Centrale e servizi applicativi) che implementa l'elaborazione e l'esposizione delle funzionalità.

Questa separazione è anche fisica perché ogni blocco viene eseguito in Pod distinti (con Service per discovery via DNS interno, es. `influxdb-service` e `mosquitto-service`), così che il layer logico non dipenda da IP/staticità ma solo da endpoint di servizio.

Il broker disaccoppia produttori e consumatori: il layer logico può scalare o riavviarsi senza “spezzare” l'ingest, mentre InfluxDB resta un componente stateful con volume persistente (PVC) e strategia di avvio che privilegia consistenza dei dati.

Gli “script” Python non vengono eseguiti come processi manuali sui nodi: Kubernetes li avvia come processo principale del container tramite il command del Deployment (un container = un servizio, oppure più container nello stesso Pod quando serve co-locazione, ad esempio per condividere dati/volumi e ridurre latenza).

Lo scheduling decide su quale nodo far girare ciascun Pod; la replica (campo `replicas`) permette di distribuire più istanze di uno stesso microservizio su nodi diversi per au-

mentare throughput e tolleranza ai guasti, mantenendo invariata l'interfaccia grazie ai Service.

2.2 Simulazione del Mondo Fisico (Digital Twin)

Nel progetto i digital twin di droni e client sono implementati come generatori controllati di telemetria e domanda, utili a validare l'architettura end-to-end senza dipendere da dispositivi fisici.

Il twin del drone (in `drone_simulator.py`) mantiene uno stato interno minimale ma realistico (posizione, batteria, usura e stato operativo) che evolve a tick: pubblica periodicamente la telemetria sul broker MQTT (topic `telemetry/drones`) e si mette in ascolto di comandi sul proprio canale dedicato comandi `DRONE_ID`, reagendo a events come assegnazione missione o rientro alla base e simulando consumi/degrado fino a condizioni di `MAINTENANCE`.

In parallelo, il twin del client (in `client_simulator.py`) genera carico “business” pubblicando nuovi ordini su MQTT (topic `business/ordini/nuovi`) con intervalli casuali e payload strutturati (coordinate pickup/drop, peso, priorità), così da stressare broker e layer applicativo in modo riproducibile.

Dal punto di vista infrastrutturale, entrambi vengono eseguiti su Kubernetes come Deployment separati, ciascuno nel proprio Pod (configurati nei manifest `configs/drone.yaml` e `configs/client.yaml`): il processo principale del container è lo script Python definito nel command, e la connettività verso l'infrastruttura è ottenuta via service discovery (`mosquitto-service`).

La “moltiplicazione” della flotta è ottenuta in modo nativo tramite lo scaling: aumentando `replicas` del Deployment `drone-simulator` Kubernetes crea più Pod identici, e poiché `DRONE_ID` viene derivato dal `metadata.name` del Pod, ogni replica corrisponde a un nuovo drone digitale con identità e canale comandi distinti.

Questo scaling può essere attivato anche in modo automatico dall'agente: attraverso il server MCP viene esposto il tool `'scale_drone_deployment'` che invoca la patch della risorsa `'deployment/scale'` (equivalente concettuale di un `'kubectl scale'`), con guardrail operativi per mantenere controllato l'impatto delle azioni automatiche sul cluster 'vedi capitolo 4'.

2.3 Osservabilità (Observability Strategy)

Come da direttive del corso, un sistema distribuito deve permettere una chiara visibilità operativa per consentire il monitoraggio dello stato, la tracciabilità delle decisioni agentiche e la rapida mitigazione dei guasti. Nel nostro progetto una strategia di osservabilità basata su tre pilastri fondamentali: la visualizzazione real-time tramite una Dashboard Web centralizzata, il tracciamento di log strutturati per la tracciabilità delle azioni e la persistenza di metriche temporali.

1. Dashboard Web Centralizzata

La visibilità generale dello stato del sistema è affidata al server centrale (`central_server.py`), il quale ospita un'interfaccia utente web integrata tramite template HTML asincroni e un set di endpoint REST nativi in Flask:

- **/api/status:** Fornisce un riepilogo numerico aggregato (*Snapshot*) del sistema, permettendo la visualizzazione del conteggio totale dei droni attivi, degli ordini pendenti e delle consegne completate, oltre alla configurazione dei nodi di rete (broker MQTT e istanza InfluxDB).
- **/api/orders:** Espone la coda dei messaggi in stato **PENDING** memorizzati in RAM.
- **/api/drones:** Restituisce la matrice completa dei droni agganciati al cluster, visualizzandone dinamicamente lo stato e le altre varie metriche.
- **/api/completed:** Elenca lo storico logistico delle consegne andate a buon fine.

Lato client, la dashboard esegue un ciclo di *polling* asincrono tramite chiamate **fetch()** JavaScript ogni 5000 millisecondi (5 secondi). Questo intervallo garantisce un aggiornamento visivo costante.

2. Log di Sistema e Audit Trail dei Comandi Agentici

Il tracciamento dei log è strutturato su più livelli per garantire l'ispezionabilità (*Auditability*) richiesta.

1. **Registro di Audit Strutturato (audit_actions.jsonl):** Gestito nativamente dal modulo `drone_mcp_layer.py`, archivia in formato JSON Lines ogni singola operazione di scrittura o rimediatazione avviata dall'IA. Traccia timestamp, il tipo di azione chiamata e l'esatto payload.
2. **Registro delle Approvazioni Umane (pending_approvals.jsonl):** Traccia il ciclo di vita delle richieste degli agenti quando riconoscono il superamento dei confini di policy. Il file contiene la richiesta iniziale dell'agente, la giustificazione logica fornita dall'LLM e il successivo cambiamento di stato causato dall'operatore.
3. **Log di Runtime e Tracciamento dei Costi:** I simulatori dei droni stampano a terminale log continui riguardanti il loro stato. Parallelamente, l'orchestratore e gli agenti loggano per ciascuna iterazione l'esatto consumo di token restituito dai modelli. Questa metrica loggata è cruciale per la verifica dei tetti di spesa.

3. Metriche di Serie Temporalì su InfluxDB

L'ultimo pilastro dell'osservabilità riguarda l'archiviazione e la persistenza dei dati operativi sotto forma di serie temporali. Quando il server centrale intercetta i messaggi sui topic MQTT, scrive in modalità sincrona record all'interno del bucket `iot_data`:

- **Measurement drone_telemetry:** Registra l'evoluzione fisica della flotta.
- **Measurement business_orders:** Monitora il flusso di ordini in ingresso.

Tramite il layer MCP (`get_drones_telemetry` e `get_pending_orders`), gli LLM possono analizzare finestre temporali passate (*time-windows* di 5 o 60 minuti) per estrarre trend di carico o pattern di deterioramento hardware.

Capitolo 3

Agentic AI e Design MCP (Model Context Protocol)

3.1 Il Limite dell'Automazione Deterministica

Normalmente l'assegnazione degli ordini ai vettori verrebbe gestita tramite un approccio deterministico basato su alberi decisionali (scripting *if-else*). Sebbene questo metodo sia efficiente per carichi di lavoro lineari, si rivela inadeguato nel dominio complesso affrontato dal nostro sistema.

Nel nostro digital twin, ogni missione è influenzata da una combinazione di variabili diverse: la priorità dell'ordine, il peso del carico, la percentuale di batteria residua, il tasso di usura del mezzo. Scrivere regole rigide per coprire ogni caso possibile porterebbe a un codice fragile e difficilmente manutenibile. Gli agenti superano questo limite sostituendo la rigidità deterministica con un ragionamento probabilistico. L'Agente valuta dinamicamente i trade-off operativi e ottimizza il matching Ordine-Drone in tempo reale, adattandosi a contesti imprevisi senza necessità di pre-programmazione esplicita.

3.2 Architettura MCP

Per garantire che l'IA operi all'interno di confini sicuri, il sistema implementa un'architettura basata sul *Model Context Protocol* (MCP). L'MCP funge da strato tra i loop di ragionamento degli LLM (confinati in `logistic_ai_brain.py`) e l'infrastruttura fisica simulata su Kubernetes. L'LLM non possiede credenziali dirette o permessi di root sui database; comunica esclusivamente tramite richieste HTTP al server MCP, il quale espone due categorie di *Tools*:

- **Observer Tools (Read-Only):** Strumenti sicuri utilizzati dall'agente per la fase di raccolta delle informazioni necessarie. Includono `get_drones_status` (interrogazione API Kubernetes), `get_drones_telemetry` e `get_pending_orders` (estrazione dati in time-window da InfluxDB).
- **Remediation Tools (Write):** Strumenti attivi che modificano lo stato del sistema. Includono `send_mqtt_command` per l'assegnazione delle missioni e `scale_drone_deployment` per l'Auto-Scaling dei pod. Essendo potenzialmente pericolosi, la loro esecuzione all'interno del modulo `drone_mcp_layer.py` è subordinata a policy che vedremo successivamente

3.3 Ruolo 1: The Health Agent

L'**HealthAgent** si occupa della resilienza dell'infrastruttura, della diagnostica della flotta e del dimensionamento delle risorse fisiche. L'agente opera analizzando i flussi di telemetria e agisce come un supervisore automatizzato della salute del sistema, gestendo monitoraggio, triage dell'usura e auto-scaling.

Compiti principali

l'agente analizza in modo proattivo il rapporto tra gli ordini pendenti e la capacità attuale della flotta. Se rileva un collo di bottiglia, decide a quante unità, e di conseguenza può Kubernetes, scalare il cluster per mantenere inalterati gli SLO. In alcuni casi che vedremo successivamente richiede direttamente l'intervento umano.

Inoltre, Il ciclo di ragionamento dell'agente include l'analisi logica per l'identificazione di guasti (es. esaurimento improvviso della batteria in volo o schianti). Se un drone si trova in stato di **MAINTENANCE** ma le sue coordinate spaziali differiscono da quelle dell'HUB centrale $[0.0, 0.0]$, l'**HealthAgent** deduce autonomamente che il mezzo è precipitato sul campo e innesca le procedure di recupero.

3.4 Ruolo 2: The Logistic Agent

Il **LogisticAgent** si occupa della logica di business del sistema, gestendo la coda degli ordini asincroni inviati dal client simulator e coordinando l'effettiva assegnazione delle missioni sui singoli mezzi.

Compiti principali

L'agente logistico incrocia i dati degli ordini con lo stato telemetrico real-time fornito dai droni che si trovano in stato **IDLE**. L'assegnazione dell'ordine non avviene secondo criteri sequenziali standard (es. FIFO), ma attraverso una ponderazione del rischio:

1. **Analisi dell'ordine:** Il peso del pacco (**weight_kg**) influisce direttamente sulle performance del drone: un pacco pesante riduce la velocità di crociera e incrementa il consumo di batteria.
2. **Contesto Spaziale:** La distanza euclidea impone un calcolo preventivo dell'autonomia necessario al viaggio.
3. **Stato del Mezzo:** Il livello di batteria attuale e il grado di usura meccanica (*wear*) accumulato.

Se un ordine presenta priorità elevata (*high priority*) ma un peso prossimo al limite massimo di 5.0 kg, l'agente eviterà tassativamente di assegnarlo a droni non in condizioni ottimali (es. $> 80\%$) o con poca batteria, riducendo le probabilità di malfunzionamenti. Una volta cercata la coppia ottimale Ordine-Drone, l'agente lancia il comando `MCP send_mqtt_command` scatenando la partenza.

3.5 Orchestrazione Dinamica

Il coordinamento e l'attivazione degli agenti sono gestiti dall'orchestratore centrale (`logistic_ai_brain`). Per garantire la massima efficienza economica, (*Budget Compliance*) di cui parleremo in seguito, l'orchestratore utilizza un'architettura reattiva, basata sul contesto.

Si articola in tre meccanismi chiave implementati in `logistic_ai_brain.py`:

1. **Triage Manager e Context Injection:** Gli agenti non sprecano più interazioni inutili. È l'orchestratore che agisce da router (`triage_manager`): analizza la fotografia del sistema e decide se serve l'intervento dell'Health Agent o del Logistic Agent. Una volta stabilito il target, l'orchestratore filtra i dati (fornendo alla logistica, ad esempio, *solo* i droni IDLE) e li inietta direttamente nel prompt iniziale dell'agente (*Context Injection*). Questo costringe l'LLM a passare immediatamente alla fase di invocazione dei tool di rimediazione.
2. **Esecuzione Parallela Asincrona:** Qualora il *Triage Manager* stabilisca che sia necessaria sia un'azione di manutenzione/scaling che un'assegnazione di ordini, i due agenti non vengono più eseguiti in modo sequenziale. L'orchestratore li istanzia e li avvia parallelamente su processi separati (`threading.Thread`), facendoli successivamente ricongiungere. Questo approccio asincrono dimezza la latenza del ciclo decisionale, permettendo al sistema di rispondere in modo molto più reattivo a picchi di traffico imprevisti.

Capitolo 4

Operational Safety Policy e Agent Guardrails

L'introduzione di layer come MCP espone il sistema a rischi intrinseci legati ad allucinazioni, latenze e comportamenti non deterministici. Per questo sono state inserite Safety Policy e vincoli nell'architettura, implementati per garantire che ogni azione degli agenti rimanga verificabile, confinata e sicura.

4.1 Principio del Minimo Privilegio (Least Privilege)

Seguendo specifiche di progetto, abbiamo fatto sì che l'architettura applichi il Principio del Minimo Privilegio (*Least Privilege*): l'Intelligenza Artificiale è stata utilizzata per ricoprire il ruolo di "assistente operativo" e non può in alcun caso avere il controllo illimitato dell'infrastruttura.

Per implementare questo livello di sicurezza, i cicli di ragionamento dei due agenti sono stati confinati all'interno di un ambiente di esecuzione isolato gestito dall'orchestratore. Gli agenti non possiedono credenziali di root, né token diretti ai cluster Kubernetes o al database InfluxDB. L'unica interfaccia col mondo reale è il server Model Context Protocol, il quale agisce come *chokepoint* di sicurezza applicando le seguenti regole:

- **Accesso Read-Only alla Telemetria:** Conformemente alle linee guida, la telemetria operativa viene esposta esclusivamente in modalità di sola lettura. Gli strumenti di osservazione implementano sotto il cofano solo `query_api` pre-compilate in Python. L'agente non ha facoltà di comporre query *Flux* o *SQL* arbitrarie, annullando matematicamente il rischio di *Injection*.
- **Divieto Assoluto di Azioni Distruttive:** Non è stato esposto alcun tool che permetta la compromissione dei dati primari. Azioni ad alto rischio come il **DROP** di un bucket, l'operazione di **DELETE** di un ordine storico su InfluxDB, o la terminazione dei processi core sono quindi fisicamente impossibili per l'agente. Il codice del layer MCP (`drone_mcp_layer.py`) omette intenzionalmente qualsiasi implementazione logica di questi comandi.

Questo crea quindi un perimetro di sicurezza: anche nel caso di una forte allucinazione logica in cui l'LLM tentasse autonomamente di cancellare i dati operativi o distruggere il cluster, le richieste verrebbero rigettate vista l'inesistenza del metodo, preservando la sicurezza del sistema.

4.2 Protezione contro Loop e Consumo Token

Una comune vulnerabilità degli agenti è la possibilità che entrino in un loop di ragionamento infinito. Questo accade quando il modello subisce un'allucinazione e di conseguenza continua a provare ad utilizzare le stesse chiamate API senza però mai giungere a una conclusione. Oltre a danneggiare il sistema, questo porterebbe anche ad un esaurimento del budget di token a disposizione. L'health agent dispone quindi di un contatore che tiene traccia delle chiamate effettuate all'interno dello stesso turno, impostato a 3 iterazioni. Se l'agente non riesce a completare il suo ragionamento in questo limite, viene terminato forzatamente il processo e restituito un messaggio predefinito. In più è presente anche un altro controllo che monitora invece il format della risposta, e qualora l'agente abbia "allucinazioni" e la risposta non rientri nei parametri appropriati per due volte consecutive, viene bloccato allo stesso modo. In questo modo limitiamo l'ingresso in cicli infiniti, assicurando che le nostre spese rimangano all'interno del budget.

4.3 Protezione contro scollegamento backend

È stato inoltre implementato un interruttore di sicurezza per impedire lo spam di chiamate API verso l'LLM quando l'infrastruttura di backend va offline. Infatti qualora il server MCP si disconnettesse, la funzione di recupero dello stato globale fallirebbe in modo sistematico. Il codice Python continuerebbe a far girare il loop ogni pochi secondi inviando all'intelligenza artificiale contesti vuoti o dati obsoleti, costringendo gli agenti a prendere decisioni basate sul nulla e generando centinaia di invocazioni API a vuoto. Per evitare questo spreco di token e risorse, ogni chiamata fallita verso il server, incrementa un contatore dedicato. Al raggiungimento di una soglia prefissata, scatta automaticamente un kill-switch che congela l'attività degli agenti. Una volta attivata questa sospensione globale, il sistema procederà a effettuare un controllo della rete a intervalli con lo scopo di verificare il ripristino del backend. Non appena il server MCP torna correttamente online, il contatore viene azzerato, l'interruttore di sicurezza si riarma e il flusso operativo dell'intelligenza artificiale riprende autonomamente la sua normale esecuzione in tempo reale.

4.4 Idempotenza delle Azioni

In questo tipo di architetture, l'idempotenza rappresenta una proprietà di sicurezza fondamentale per garantire la robustezza del sistema e prevenire fallimenti catastrofici causati da allucinazioni o comportamenti ripetitivi degli LLM. Un'operazione è idempotente quando il suo effetto sul sistema è lo stesso, sia in caso venga eseguita una volta sola, sia in caso venga eseguita più volte.

L'esempio più significativo di questa protezione è verificabile nel meccanismo di scaling della flotta. Quando l'agente della salute determina la necessità di espandere il numero di droni e invoca lo strumento `scale_drone_deployment`, il codice nel modulo `drone_mcp_layer.py` esegue un'operazione di *patching* sullo stato del cluster tramite la funzione `patch_namespaced_deployment_scale`. Questa scelta garantisce l'idempotenza grazie a due fattori chiave:

- **Logica di Stato Desiderato (Dichiarativa):** Il tool accetta come parametro il numero di repliche desiderate (ad esempio 6 repliche). Se l'LLM entra in un loop o ripete lo stesso comando cinque volte nello stesso ciclo, Kubernetes riceverà cinque richieste consecutive in cui si dichiara che lo stato ottimale è di 6 pod. Il pannello di controllo di Kubernetes verificherà che il deployment ha già raggiunto la quota di 6 repliche e ignorerà completamente le successive quattro chiamate, non eseguendo alcuna allocazione extra.
- **Prevenzione del Collasso per Esaurimento Risorse:** Se il tool MCP fosse stato progettato in modo da aggiungere un numero di mezzi ad ogni chiamata, cinque chiamate consecutive dettate da un'allucinazione avrebbero inserito molti più droni fantasma nel cluster. Questo avrebbe saturato istantaneamente le risorse computazionali dei nodi congestionando la rete MQTT e violando il tetto massimo di spesa.

4.5 Human-In-The-Loop (HITL) ed Escalation

Il paradigma *Human-In-The-Loop* (HITL) rappresenta l'ultimo livello di difesa contro allucinazioni o decisioni economicamente dannose generate autonomamente dall'Intelligenza Artificiale. Nel nostro sistema, l'automazione pura viene interrotta non appena un'azione degli agenti supera i confini di sicurezza prestabiliti dalle policy, forzando un'escalation verso un supervisore umano.

La Regola Architetture dei 6 Droni

All'interno del modulo `drone_mcp_layer.py`, la costante `POLICY_LIMITS` fissa a **6** il numero massimo di droni allocabili per l'azione in totale autonomia (`max_drones_auto`). Il flusso operativo è regolato da una policy rigorosa:

- **Sotto la soglia (≤ 6):** L'HealthAgent ha il pieno controllo dichiarativo e può invocare direttamente il tool `scale_drone_deployment` per adattare il cluster Kubernetes ai picchi di carico.
- **Sopra la soglia (> 6):** Qualora un workload maggiore richieda un dimensionamento superiore a 6 pod, il layer MCP blocca l'esecuzione deterministica del comando, considerandolo ad alto impatto economico. L'agente è obbligato a interrompere il proprio loop e a invocare lo strumento `request_human_approval`.

Il Portale di Approvazione

L'intercettazione e la gestione delle escalation sono demandate a un servizio Flask indipendente (`human_approval_manager.py`) in ascolto sulla porta 5002, che espone una Dashboard Web dedicata agli operatori umani.

Quando l'agente richiede un'escalation, il server MCP persiste la richiesta nel file `pending_approvals.jsonl`. La dashboard legge questo registro e renderizza una scheda informativa per l'operatore. Ciascuna scheda mostra in modo trasparente l'identificativo unico (`request_id`), l'azione richiesta, il payload JSON (es. `{'replicas': 8}`) e, soprattutto, la giustificazione logica in linguaggio naturale fornita dall'LLM (`reason`).

L'operatore dispone di un diritto di veto assoluto tramite due interfacce. Approve fa sì che la transizione venga convalidata, ed esegue istantaneamente il *patching* su Kubernetes impostando il parametro **force=True**, scavalcando i blocchi preventivi del codice. Deny contrassegna la richiesta come rejected e al ciclo successivo l'health agent manterrà lo stato inalterato.

Capitolo 5

Meccanismi di Reliability e Scalabilità

Il presente capitolo illustra le soluzioni applicate all'interno del progetto per gestire temporanei disservizi e scalare le risorse al bisogno. L'architettura fa ampio uso dei comportamenti "out-of-the-box" forniti dalle librerie di base, arricchite da pattern architetturali per confinare gli errori e rendere il sistema reattivo al traffico.

5.1 Resilienza di Rete

Nei sistemi distribuiti basati su messaggistica (come i client in `drone_simulator.py`), la sconnessione temporanea del broker MQTT è un'evenienza da gestire nativamente.

In fase di avvio (funzione `run()` del simulatore), i droni presentano un ciclo di retry custom: qualora il broker non sia ancora schedulato (es. deployment asincrono in un cluster Kubernetes), la connessione fallisce ed esegue un poller ritardato di 5 secondi tramite `time.sleep`, fino ad avvenuto handshake.

A regime, il grosso della resilienza di rete è demandato ai meccanismi interni della libreria ufficiale *paho-mqtt* e al suo metodo associato `loop_start()`. In caso di caduta momentanea del servizio *Mosquitto*, il client Python cattura il blocco della socket ed avvia autonomamente una strategia di Exponential Backoff per i tentativi di riconnessione, ripristinando la sottoscrizione ai topic operativi (`comandi/#`) senza introdurre logiche custom che rischierebbero di sovraccaricare il thread principale.

5.2 Graceful Degradation del Server Centrale

Nel nostro ecosistema, `central_server.py` gestisce sia le telemetrie (fino a InfluxDB) sia l'esposizione della dashboard Flask in tempo reale per la supervisione della flotta. Per evitare che un downtime limitato del database InfluxDB causi il collasso del monitoraggio live, è stato adottato un approccio ibrido con degradazione controllata.

Il pattern prevede un dual write all'arrivo di ogni messaggio MQTT (`on_message`), strutturato secondo la seguente logica sequenziale:

```
1 def on_message(client, userdata, msg):
2     # 1. Aggiornamento immediato dello stato in-memory (per la
    Dashboard live)
3     state["drones"][drone_id] = parsing(msg.payload)
```

```

4      # 2. Scrittura persistente su InfluxDB isolata da eccezioni
5      try:
6          write_api.write(bucket, org, record)
7      except InfluxDBError as e:
8          # L'eccezione viene neutralizzata per preservare i
9      servizi realtime
10         log_warning("InfluxDB non raggiungibile. Degradazione del
            servizio log storici.")

```

Listing 5.1: Schema logico del meccanismo di Graceful Degradation nel Server Centrale.

Se il container InfluxDB risulta non accessibile, l'eccezione viene neutralizzata nel blocco `try/except` globale all'handler. Di conseguenza, il sistema perde il salvataggio dei log storici (degradazione del servizio) ma mantiene perfettamente operativi il tracciamento realtime dei pacchetti e i cruscotti per gli operatori (graceful), isolando il guasto al solo storage layer senza bloccare la coda delle missioni.

5.3 Scalabilità Dinamica e Safety Limits K8s

L'architettura supera lo scaling passivo (es. basato unicamente su carico CPU/RAM) delegando la valutazione dei bisogni logistici ad Agenti AI dotati di capacità MCP (Model Context Protocol).

Nello specifico, il *Logistic AI Brain* ha a disposizione un tool per richiedere dinamicamente più droni per snellire la coda (`pending_orders`). Internamente, questo strumento richiama una funzione dedicata in `drone_mcp_layer.py` (`scale_drone_deployment`) che interagisce via API HTTP con il control-plane di Kubernetes. Sfruttando la libreria client ufficiale per Python, la richiesta genera un'operazione K8s di tipo `patch_namespaced_deployment_scale` a carico del Deployment `drone-simulator`, variando in tempo reale le repliche a disposizione a seconda della coda dei pacchi.

A tutela dell'infrastruttura (*Operational Safety*), questo scaling dinamico è sottoposto a policy di sicurezza:

```

1  # Verifica dei vincoli architetturali prima del patching su
    Kubernetes
2  if requested_replicas > POLICY_LIMITS:
3      # L'azione viene bloccata ed escalata all'operatore umano
4      trigger_human_escalation(requested_replicas)
5  else:
6      # Esecuzione della patch sul Deployment
7      k8s_client.patch_namespaced_deployment_scale(name="drone-
        simulator", ...)

```

Listing 5.2: Guardrail di sicurezza applicato alle richieste di scaling dell'agente.

Se l'applicativo tenta un'espansione del numero di repliche oltre la soglia prestabilita `POLICY_LIMITS`, l'azione viene intercettata, bloccata ed escalata al *Human Approval Manager*, evitando consumi cloud incontrollati dovuti a potenziali allucinazioni o comportamenti imprevisti del modello.

Capitolo 6

Chaos Engineering e Test di Fallimento

Per validare empiricamente i meccanismi di reliability descritti nel Capitolo 5, è stata realizzata una suite di chaos engineering (`ops/chaos_test_suite.py`) che inietta in modo controllato guasti rappresentativi sui componenti più critici dell'architettura (database, broker, capacità di flotta). L'obiettivo non è dimostrare l'assenza di guasti, ma misurare la *reazione* del sistema in termini di RTO, perdita di dati e capacità di auto-rimediazione agentica.

6.1 Metodologia

Ogni esperimento è strutturato secondo il ciclo classico del chaos engineering:

1. **Ipotesi:** definizione del comportamento atteso (SLO da rispettare).
2. **Blast radius:** il guasto è confinato a un singolo Deployment in namespace `default`, non si tocca il control-plane.
3. **Iniezione:** eseguita via API Kubernetes (`patch_namespaced_deployment_scale`, `delete_namespaced_pod`) o tramite generatore di carico (`client_simulator.py --stress`).
4. **Osservazione:** raccolta di metriche su `chaos_test_results.log` (timestamp, RTO, repliche prima/dopo).
5. **Abort condition:** timeout massimo di 60s per il ripristino, oltre il quale il test è considerato fallito.

6.2 Esperimento 1: Primary Database Outage

Ipotesi. Lo spegnimento di InfluxDB non deve interrompere il monitoraggio real-time (*Graceful Degradation*, §5.2); la dashboard deve restare reattiva e il sistema deve recuperare entro l'SLO di disponibilità del data layer (§7.1).

Procedura. Scaling del Deployment `influxdb` a 0 repliche, attesa di 10s, ripristino a 1 replica e cronometraggio del tempo necessario affinché il pod torni in stato `Ready`.

Risultato. RTO misurato: 12,05 s (run del 23/05/2026). Durante l’outage il `central_server` ha continuato a servire la dashboard sfruttando lo stato in-memory; le sole scritture storiche sono state perse, in linea col pattern di dual-write neutralizzato da `try/except`.

6.3 Esperimento 2: Network Resilience (MQTT Broker Crash)

Ipotesi. L’eliminazione forzata del pod `mosquitto` non deve compromettere la flotta: Kubernetes deve ricreare il pod e i client `paho-mqtt` devono riconnettersi autonomamente tramite *exponential backoff* (§5.1).

Procedura. Deletion del pod via `delete_namespaced_pod` e attesa del nuovo pod in stato `Running`.

Risultato. Downtime di scheduling K8s: 0,04 s (rischedulazione immediata sul nodo). La riconnessione dei droni è gestita dal client MQTT senza logica custom; il broker non perde messaggi grazie alla natura at-most-once della telemetria (perdite tollerate, frequenza 2 s).

6.4 Esperimento 3: Load Spike e Scaling Agentico

Ipotesi. L’iniezione simultanea di 50 ordini deve essere rilevata dal `triage_manager` e tradursi in una richiesta di scaling dell’`HealthAgent`; sotto la soglia di 6 droni lo scaling avviene in autonomia, oltre tale soglia parte l’escalation HITL (§4.5).

Procedura. Lettura delle repliche correnti del Deployment `drone-simulator`, esecuzione di `client_simulator.py --stress` (durata effettiva ≈ 56 s), attesa ulteriore di 30 s per consentire al `LogisticAIBrain` di completare un ciclo di triage, riletture delle repliche.

Risultato osservato. Repliche **prima = 3, dopo = 6**. L’orchestratore agentico ha rilevato il backlog tramite query InfluxDB, invocato il tool MCP `scale_drone_fleet` e raddoppiato la flotta entro ≈ 86 s end-to-end dalla prima iniezione. Il test conferma l’efficacia dello scaling agentico per shock di carico controllati su orizzonti dell’ordine del minuto; per shock sub-minuto resta aperta la traccia (§6.6) di un HPA Kubernetes nativo come *fast-path* complementare.

6.5 Riepilogo Metriche

Esperimento	Metrica	Valore
InfluxDB outage	RTO	12,05 s
MQTT broker crash	Downtime K8s	0,04 s
Load spike (50 ord.)	Δ repliche	$3 \rightarrow 6$ (≈ 86 s)

Tabella 6.1: Risultati misurati eseguendo `chaos_test_suite.py` (run del 23/05/2026).

6.6 Lezioni Apprese

Gli esperimenti confermano la robustezza dei livelli infrastrutturali (Kubernetes + paho-mqtt + dual-write) e validano lo strato agentico come meccanismo di scaling efficace su orizzonti di ≈ 1 min. Restano due limiti aperti: (i) la latenza del ciclo decisionale (≈ 30 s dominati da `time.sleep(30)` e inferenza LLM) è ancora elevata per spike sub-minuto, dove un HPA Kubernetes nativo offrirebbe un *fast-path* complementare; (ii) la suite non misura ancora la *loss rate* di telemetria durante l'outage di InfluxDB né l'effettiva latenza end-to-end di assegnazione ordini sotto stress. La convivenza HPA + agente (HPA come reattore rapido, agente come pianificatore strategico) rientra tra gli sviluppi futuri (Cap. 8).

Capitolo 7

Analisi Economica: Costi, ROI e ROA (Return on Agent)

7.1 Service Level Objectives (SLOs) e Costo del Downtime

Definizione degli SLOs

Per analizzare l'affidabilità del sistema e definire la tolleranza ai guasti, abbiamo stabilito tre *Service Level Objectives* (SLO), basandoci sul workload visto nei capitoli precedenti.

- **SLO 1 - Latenza di Assegnazione Logistica:** Il 90% degli ordini arrivati con priorità alta deve essere processato e assegnato ad un mezzo entro 1 minuto. Questo valore è giustificato dal ciclo di vita dell'orchestratore, che prevede un `time.sleep(30)` di base a cui si somma il tempo di latenza per l'inferenza dell'API.
- **SLO 2 - Disponibilità del Data Layer:** Il broker MQTT e il database InfluxDB devono garantire il 99.9% di uptime. Considerando che i droni inviano telemetria ogni 2 secondi, una perdita prolungata di dati (comunque mitigata dalle tecniche che abbiamo visto) renderebbe cieca l'IA, paralizzando l'health agent nella sua funzione di monitoraggio con possibili risvolti pericolosi.
- **SLO 3 - Affidabilità e Risoluzione Agentica:** L'LLM deve concludere il proprio task di ragionamento ed emettere una *tool call* valida nel 99% dei casi senza innescare l'interruttore di sicurezza `MAX_AGENT_ITERATIONS`, evitando timeout o spreco di computazione e risorse economiche.

Stima del Costo di un'Ora di Downtime

Lavorando nel dominio della logistica critica, il costo stimato di un'ora di downtime del servizio è stimato a circa **15.000 €/ora**. Questa stima deriva da una media calcolata basandosi sui guadagni medi che il servizio raggiunge in un'ora di attività, portando a termine le consegne. Il dato però deriva anche da possibili penali contrattuali, e quindi dagli SLA violati.

7.2 Valutazione del Return on Agent (ROA) e Costi

L'orchestrazione autonoma della flotta introduce un bilanciamento critico tra i costi operativi delle API (OPEX) e il risparmio di capitale hardware (CAPEX) ottenuto tramite logiche predittive.

7.2.1 Strategie di Ottimizzazione dei Costi Operativi (OPEX)

Il sistema limita l'impatto economico delle chiamate LLM attraverso tre strategie integrate:

- **Triage Preventivo:** Il modulo `trriage_manager` analizza lo snapshot del sistema prima di invocare i modelli, attivando `LogisticAgent` ed `HealthAgent` solo se sussistono reali condizioni di sbilanciamento (es. ordini pendenti accoppiati a droni idle, o necessità di scaling/manutenzione). Questo approccio azzerava i costi derivanti da cicli di no-op ridondanti.
- **Ingegneria dei Prompt:** I prompt di sistema impongono restrizioni sintattiche che vietano la generazione di testo libero, preamboli o spiegazioni discorsive del ragionamento. L'output dei modelli è rigidamente confinato alle sole chiamate di funzione (tool calling), minimizzando drasticamente il consumo di token di completamento e azzerando le computazioni superflue.
- **Monitoraggio dei Consumi:** Il sistema esegue un auditing sistematico dei token spesi (prompt, completamento e totale) al termine di ogni esecuzione, permettendo una precisa rendicontazione finanziaria del processo decisionale.

7.2.2 Mitigazione del Rischio e Preservazione del Capitale (CAPEX)

Il ritorno sull'investimento dell'agente intelligente si concretizza nella mitigazione dei rischi d'impresa e nell'ottimizzazione infrastrutturale:

- **Manutenzione Preventiva Dinamica:** `LogisticAgent` applica la regola algoritmica di escludere droni con usura superiore al 70% dal trasporto di carichi pesanti (> 3 kg). Nel contesto fisico del simulatore, questa decisione previene l'esaurimento imprevisto della batteria in volo e la conseguente perdita catastrofica del vettore (crash).
- **Elasticità Infrastrutturale Cloud:** L'`HealthAgent` modula dinamicamente le repliche del deployment `drone-simulator` su Kubernetes basandosi sulle fluttuazioni del carico di lavoro effettivo, riducendo i costi di over-provisioning dell'infrastruttura cloud.
- **Operational Safety Cap:** Le policy di sicurezza impongono un limite massimo di 6 droni per lo scaling automatico. Qualsiasi espansione ulteriore viene intercettata e subordinata ad approvazione umana (`request_human_approval`), ponendo un vincolo rigido (budget cap) contro anomalie o loop di scaling incontrollati.

- **Massimizzazione del Throughput Remunerativo:** La coda degli ordini viene ordinata preventivamente per priorità e valore economico. L'agente logistico alloca la capacità di carico residua partendo dai task a più alto rendimento, ottimizzando il fatturato per singola finestra operativa.

Proiezioni Economiche e Budget Compliance

Al fine di convalidare la sostenibilità finanziaria del software, è stata effettuata una stima comparativa dei costi su base mensile, ipotizzando una frequenza di esecuzione standard di 2.880 cicli giornalieri (un polling di verifica ogni 30 secondi).

Voce di Costo	Scenario Gemini 2.5 Pro	Scenario Mistral Small
Infrastruttura Cloud (3 nodi Kubernetes)	150,00 \$ / mese	150,00 \$ / mese
Token Modello Linguistico (AI)	~500,00 \$ / mese	~10,00 \$ / mese
OPEX Totale Mensile	~650,00 \$ / mese	160,00 \$ / mese

- **Costi Infrastrutturali Fissi:** Il mantenimento dell'ambiente containerizzato in alta affidabilità (comprensivo di 3 nodi worker per Kubernetes, database InfluxDB per la telemetria e broker MQTT per la messaggistica) genera un costo operativo fisso di circa **150 \$/mese**.
- **Costi Variabili dell'Intelligenza Artificiale:** L'impatto della componente generativa è fortemente condizionato dall'architettura del modello scelto. L'adozione di *Gemini 2.5 Pro* comporta un volume medio di 1.200 token per ciclo (ripartiti equamente tra input e output), traducendosi in una spesa di circa **500 \$/mese**. Al contrario, l'impiego di un modello specialistico più leggero come *Mistral Small* riduce l'impronta a 800 token per ciclo; grazie a un'architettura asimmetrica focalizzata sull'input (90% input, 10% output), il costo computazionale si abbatte a meno di **10 \$/mese**.

Considerando lo scenario basato sul modello top-tier (*Gemini 2.5 Pro*), l'OPEX complessivo del software — dato dalla somma dei costi infrastrutturali e dei token — si attesta su un valore massimo di circa **650 \$/mese**. Tale cifra rappresenta una spesa ampiamente sostenibile e prevedibile, garantendo la piena *Budget Compliance* rispetto ai vincoli economici di progetto. L'adozione di politiche di caching, l'uso di *Mistral Small* o la modulazione del ciclo di polling offrono ampi margini per un'ulteriore riduzione dei costi operativi.

7.3 Analisi dei Trade-Off

Bilanciamento analitico tra i costi di automazione e la reliability, confrontando l'efficienza dell'automazione pura rispetto alla sicurezza del paradigma HITL.

Capitolo 8

Conclusioni

Riflessioni finali sul paradigma Agentico implementato nelle infrastrutture critiche, analisi delle limitazioni attuali (ad esempio, la latenza di ragionamento degli LLM durante i picchi estremi di traffico) e prospettive per sviluppi futuri.